



OAT 2

Web API da MecâniQA: Engenharia de Software Aplicada

Modelagem de Domínio, Padrões Builder & Singleton e Entidade Associativa

Autores:

Alex Gabriel Alves dos Santos

Gustavo Azevedo Diniz

Gustavo Muniz dos Anjos

João Paulo Rosa Batista

Matheus Silva do Monte

Agenda

Abordagem de Engenharia

Apresentar a resolução dos desafios reais de negócio da oficina MecâniQA através de engenharia orientada a objetos: identificação do problema prático, proteção de integridade com o padrão Builder, desacoplamento relacional por entidade associativa, persistência em memória via Singleton e fluxo decisório do Controller.

1. O Problema da Oficina

Desafios de orquestração do atendimento, agregação de mão de obra e insumos e ciclo de vida.

2. Padrão Builder na Prática

Construção fluente em código, construtores privados e eliminação de instâncias inconsistentes.

3. Entidade Associativa

Modelagem de ItemPedidoPecas em código: repetição de peças e congelamento de preço histórico.

4. Padrão Singleton em Código

Implementação em código real do repositório em memória e injeção sem frameworks no Controller.

5. Diagrama de Atividades (US03)

Visão lógica do Controller no endpoint POST /api/pedidos com validações e respostas HTTP.

1. Qual problema prático da oficina estamos resolvendo?



Gargalo 1: Fluxo Incompleto

Orquestração do Atendimento

Cenário na MecâniQA: Na OAT 1, existiam apenas cadastros isolados de Peça e Serviço. O mecânico não conseguia abrir uma Ordem de Serviço vinculando o cliente ao veículo e aos serviços executados.

Nossa Solução: Criação da entidade agregadora `OrdemServico`, unindo múltiplos `Servico` e o `PedidoPecas` em um ciclo de vida rastreado por status estritos (ABERTO a EXECUTADO).

Gargalo 2: Instanciação Insegura

Risco de Estados Incompletos

Cenário na MecâniQA: Criar uma Ordem de Serviço por construtores tradicionais com 10 parâmetros gerava erros de digitação e permitia criar OS vazia sem cliente ou totais zerados.

Nossa Solução: Adoção do **Padrão Builder manual** com construtor estritamente `private`. O método `build()` valida as listas e aciona o cálculo financeiro seguro automaticamente.

Gargalo 3: Flutuação de Peças

Preços Voláteis e Repetições

Cenário na MecâniQA: Um veículo pode demandar a mesma peça em etapas diferentes. Além disso, se o preço da peça subir no catálogo mês que vem, a OS antiga não pode ter seu valor alterado!

Nossa Solução: Entidade Associativa `ItemPedidoPecas`, congelando o preço unitário de venda na data de inclusão e permitindo itens duplicados em lotes independentes.

2. Padrão Builder na Prática: Código Escrito e Blindagem



OrdemServico.java

```
public class OrdemServico {
    // Construtor ESTRITAMENTE PRIVADO: impede new OrdemServico()
    private OrdemServico(Builder builder) {
        this.codigo = builder.codigo;
        this.cliente = builder.cliente;
        this.veiculo = builder.veiculo;
        this.status = builder.status != null ? builder.status : StatusOrdemServico.ABERTO;
        this.servicos = builder.servicos != null ? builder.servicos : new ArrayList<>();
        this.pedidoPecas = builder.pedidoPecas;
        // Total financeiro calculado de forma imutável
        this.valorTotal = calcularValorTotal(this.servicos, this.pedidoPecas);
        this.dataAbertura = LocalDateTime.now();
    }

    public static Builder builder() {
        return new Builder();
    }
}
```

OrdemServicoTest.java

```
@Test
@DisplayName("Construtor privado deve impedir new OrdemServico()")
void deveGarantirConstrutoresPrivados() {
    Constructor<?>[] ctors = OrdemServico.class.getDeclaredConstructors();
    assertTrue(Modifier.isPrivate(ctors[0].getModifiers()));
}

@Test
@DisplayName("Instanciação fluente e segura via Builder manual")
void deveCriarOrdemServicoViaBuilder() {
    OrdemServico os = OrdemServico.builder()
        .codigo(10L)
        .cliente("Maria Oliveira")
        .veiculo("Corolla 2022")
        .servico(balanceamento)
        .build(); // Gera objeto completo e totalizado

    assertEquals(StatusOrdemServico.ABERTO, os.getStatus());
    assertEquals(new BigDecimal("60.00"), os.getValorTotal());
}
```

3. Entidade Associativa ItemPedidoPecas: Regra de Negócio e Código



```
ItemPedidoPecas.java

public class ItemPedidoPecas {
    private Long codigo;
    private Peca peca;
    private Integer quantidade;
    private BigDecimal precoUnitario;

    // Subtotal calculado com precisão monetária (BigDecimal)
    public BigDecimal getSubtotal() {
        if (precoUnitario == null || quantidade == null) return BigDecimal.ZERO;
        return precoUnitario.multiply(BigDecimal.valueOf(quantidade));
    }

    public static class Builder {
        // REGRA DE NEGÓCIO: Congela o preço no momento da inclusão
        public Builder peca(Peca peca) {
            this.pecas = peca;
            if (pecas != null && this.precoUnitario == null) {
                this.precoUnitario = peca.getPrecoVenda(); // Preço histórico
            }
            return this;
        }
    }
}
```

Garantia 1: Multiplicidade N:N Itens Duplicados ou em Lotes

Um pedido de peças pode conter a mesma peça adicionada mais de uma vez (por exemplo: 2 unidades de amortecedores dianteiros no item #1 e 2 unidades no item #2 com especificações distintas). A entidade associativa atribui uma identidade própria (codigo) ao item, permitindo essa flexibilidade sem quebrar a chave primária.

Garantia 2: Congelamento Histórico Imunidade à Inflação do Catálogo

Se uma peça cadastrada a R\$ 45,00 for reajustada amanhã para R\$ 70,00 no catálogo geral, o pedido fechado semana passada não sofrerá adulteração, pois precoUnitario armazena o valor histórico fixado no momento da venda, garantindo conformidade fiscal e contábil.

4. Padrão Singleton: Código Real e Persistência em Memória



PecaRepository.java & Controller

```
public class PecaRepository {
    // 1. Atributo estático guarda a instância única
    private static PecaRepository instance;
    private List<Peca> pecas;

    // 2. Construtor PRIVADO impede 'new PecaRepository()' externo
    private PecaRepository() {
        this.pecas = new ArrayList<>();
    }

    // 3. Método estático global de recuperação da instância única
    public static PecaRepository getInstance() {
        if (instance == null) {
            instance = new PecaRepository();
        }
        return instance;
    }
}

@RestController
public class PecaController {
    private PecaRepository pecaRepository;
    // Injeção limpa sem frameworks usando getInstance()
    public PecaController() {
        this.pecasRepository = PecaRepository.getInstance();
    }
}
```

Padrão Criacional GoF Por que Singleton no Repositório?

Como a disciplina veda o uso de bancos relacionais e anotações como `@Autowired` nesta fase, a persistência depende de listas em memória (`List<Peca>`, `List<OrdemServico>`). Se um novo repositório fosse instanciado a cada requisição HTTP, os dados cadastrados desapareceriam imediatamente.

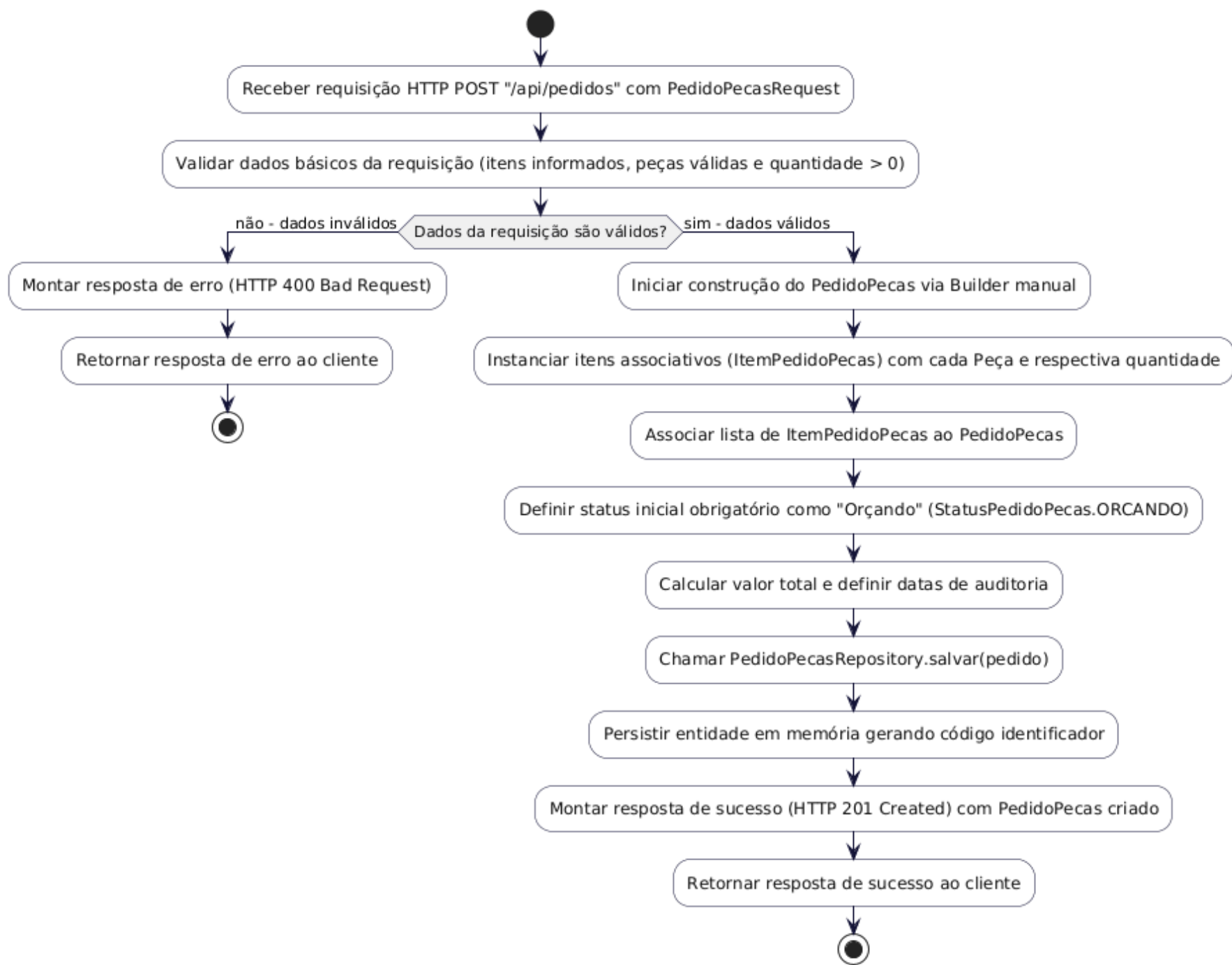
Representação no Diagrama UML

- **Estereótipo <<Singleton>>:** Identifica visualmente a classe com ciclo de vida único.
- **Membros Sublinhados:** Indicam escopo de classe estático (`instance` e `getInstance()`).
- **Acesso pelo Controller:** Dependência orientada a método estático sem acoplamento direto via `new`.

5. Diagrama de Atividade: US03 - Criar Pedido (POST /api/pedidos)



US03 - Criar Pedido de Peças - Diagrama de Atividades (Visão do Controller)



Visão Lógica do Controller

Fluxo Decisório do Endpoint

- **1. Recepção do Payload:** O Controller recebe a requisição HTTP POST /api/pedidos com o DTO PedidoPecasRequest.
- **2. Validação Defensiva:** Verifica a presença dos itens, a validade das peças solicitadas e se quantidade > 0.
- **3. Desvio Condicional (Erro):** Se os dados forem inválidos, interrompe o fluxo e retorna imediatamente HTTP 400 Bad Request.
- **4. Fluxo Principal (Sucesso):**
 - Instancia itens via ItemPedidoPecas.builder()
 - Constrói o pedido com status StatusPedidoPecas.ORDANDO
 - Calcula os subtotais e valor total automaticamente
 - Salva no repositório Singleton gerando código
 - Retorna resposta de sucesso HTTP 201 Created.

Síntese da OAT 2

Maturidade da Arquitetura

- **Problema Resolvido:** A oficina MecâniQA agora orchestra o atendimento de ponta a ponta com agregados de negócio (OS e Pedidos de Peças).
- **Encapsulamento Rígido:** Aplicação rigorosa dos padrões GoF **Builder** e **Singleton** com construtores estritamente privados blindando instâncias inconsistentes.
- **Histórico Financeiro Protegido:** A entidade associativa ItemPedidoPecas garante multiplicidade de itens e preservação histórica de preços com BigDecimal.

Fontes e Artefatos

Referências do Projeto

- **Repositório do Projeto:**
Guztaver/mecaniQA-api-rio-de-janeiro
- **Documentação e Guias:**
 - Guia de Desenvolvimento Web Orientado a Objetos - UNEX 2026
 - OAT 1 - Web API Fundacional da MecâniQA
 - Modelo de Apresentação Institucional UNEX
- **Testes Unitários Automatizados:**
 - OrdemServicoTest (construtores privados e recálculo)
 - PedidoPecasTest (entidade associativa e multiplicidade)